

tf.compat.v1.SparseConditionalAccumulator

Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/SparseConditionalAccumulator

A conditional accumulator for aggregating sparse gradients.

Function Signatures

```
tf.compat.v1.SparseConditionalAccumulator( dtype, shape=None,
shared_name=None, name='sparse_conditional_accumulator',
reduction_type='MEAN' )
```

Code Examples

```
tf.compat.v1.SparseConditionalAccumulator(
    dtype,
    shape=None,
    shared_name=None,
    name='sparse_conditional_accumulator',
    reduction_type='MEAN'
)
```

```
tf.compat.v1.SparseConditionalAccumulator(  
dtype,  
shape=None,  
shared_name=None,  
name='sparse_conditional_accumulator',  
reduction_type='MEAN'  
)
```

```
apply_grad(  
grad_indices, grad_values, grad_shape=None, local_step=0, name=None  
)
```

```
apply_grad(  
grad_indices, grad_values, grad_shape=None, local_step=0, name=None  
)
```

```
apply_indexed_slices_grad(  
grad, local_step=0, name=None  
)
```

```
apply_indexed_slices_grad(  
grad, local_step=0, name=None  
)
```

```
num_accumulated(  
name=None  
)
```

```
num_accumulated(  
name=None  
)
```

Documentation

A conditional accumulator for aggregating sparse gradients.

Inherits From: `ConditionalAccumulatorBase`

View aliases

Compat aliases for migration

See

Migration guide for
more details.

`tf.compat.v1.sparse.SparseConditionalAccumulator`

Sparse gradients are represented by `IndexedSlices`.

Up-to-date gradients (i.e., time step at which gradient was computed is equal to the accumulator's time step) are added to the accumulator.

Extraction of the average gradient is blocked until the required number of gradients has been accumulated.

Attributes

`## Methods`

`apply_grad`

[View source](#)

Attempts to apply a sparse gradient to the accumulator.

The attempt is silently dropped if the gradient is stale, i.e., `local_step` is less than the accumulator's global time step.

A sparse gradient is represented by its indices, values and possibly empty or `None` shape. Indices must be a vector representing the locations of non-zero entries in the tensor. Values are the non-zero slices of the gradient, and must have the same first dimension as indices, i.e., the `nnz` represented by indices and values must be consistent. Shape, if not empty or `None`, must be consistent with the accumulator's shape (if also provided).

`apply_indexed_slices_grad`

[View source](#)

Attempts to apply a gradient to the accumulator.

The attempt is silently dropped if the gradient is stale, i.e., `local_step` is less than the accumulator's global time step.

`num_accumulated`

[View source](#)

Number of gradients that have currently been aggregated in accumulator.

`set_global_step`

[View source](#)

Sets the global time step of the accumulator.

The operation logs a warning if we attempt to set to a time step that is lower than the accumulator's own time step.

take_grad

[View source](#)

Attempts to extract the average gradient from the accumulator.

The operation blocks until sufficient number of gradients have been successfully applied to the accumulator.

Once successful, the following actions are also triggered:

- Counter of accumulated gradients is reset to 0.
- Aggregated gradient is reset to 0 tensor.
- Accumulator's internal time step is incremented by 1.

take_indexed_slices_grad

[View source](#)

Attempts to extract the average gradient from the accumulator.

The operation blocks until sufficient number of gradients have been successfully applied to the accumulator.

Once successful, the following actions are also triggered:

- Counter of accumulated gradients is reset to 0.
- Aggregated gradient is reset to 0 tensor.
- Accumulator's internal time step is incremented by 1.